

INT8 Detection Under Degradation Report

What we did

We chose **YOLOv8n** because it runs efficiently on CPUs, includes pretrained COCO weights, and has a mature ONNX/INT8 export workflow, making it well suited for a quantization-focused study. We evaluated on a fixed **500-image subset of COCO val2017** containing five classes (person, car, bicycle, traffic light, and stop sign) with 2,353 ground-truth boxes. A separate, non-overlapping set of 120 images was used exclusively for INT8 calibration.

Task 1 - FP32 baseline, then quantize to INT8. Exported the pretrained model to a fixed 640×640 ONNX graph and measured accuracy with pycocotools.COCOeval. Then applied post-training static INT8 quantization via ONNX Runtime (QDQ format, U8S8, per-channel weights, Conv-only, quantizing the detection head's final output layer destroys every class score, so it stays in float32), calibrated on 120-image set. Compared accuracy, latency, and model size against FP32.

Task 2 - Degrade. Built four degraded copies of the same 500 images (motion blur, low light, JPEG compression, downscale/upscale) and ran both FP32 and INT8 across all four, to see whether quantization's accuracy cost gets worse under realistic image degradation.

Task 3 - one targeted intervention. Picked the worst-performing degradation (motion blur) and tried a single, targeted fix - recalibrating INT8's quantization scales on blurred images instead of clean ones, to test whether that closed the gap.

Results

Table 1. FP32 vs. INT8 - accuracy, latency, and model size (clean images).

Metric	FP32	INT8	Difference
mAP@0.5	0.586	0.569	-3.0%
mAP@0.5:0.95	0.415	0.394	-4.9%
Latency, mean (1 thread)	134.9 ms	69.7 ms	1.94x faster
Latency, p95 (1 thread)	138.6 ms	72.0 ms	1.93x faster
Model size (ONNX)	12.85 MB	3.60 MB	3.57x smaller

INT8 gives roughly a 2x speedup and 3.6x size reduction for a 3-5% relative accuracy cost. Small-object AP took largest hit (-18.9% relative) while large-object AP was essentially unaffected, 8-bit resolution has least room to spare exactly where the signal is thinnest.

Table 2. Per-class AP, FP32 vs INT8 (clean images).

Class	FP32 AP@0.5	INT8 AP@0.5	FP32 AP@0.5:0.95	INT8 AP@0.5:0.95
person	0.7751	0.7630	0.5291	0.5111
bicycle	0.4081	0.3885	0.2446	0.2356
car	0.5937	0.5879	0.3797	0.3704
traffic light	0.4361	0.4284	0.2326	0.2138
stop sign	0.7178	0.6761	0.6872	0.6411

Per-class AP shows that INT8 preserved most of YOLOv8n's detection capability, with losses generally below 0.05 AP. The largest degradation occurred for **stop signs** and **traffic lights**, suggesting that quantization error disproportionately affects classes dominated by small objects and fine visual details.

Table 3. AP by object size, FP32 vs INT8 (clean images).

Size	FP32 AP@[.5:.95]	INT8 AP@[.5:.95]	Difference
Small (<32 ² px)	0.2045	0.1658	-0.0387 (-18.9%)
Medium (32 ² -96 ² px)	0.5978	0.5730	-0.0248 (-4.1%)
Large (>96 ² px)	0.7186	0.7518	+0.0331 (+4.6%)

Small-object AP takes the largest relative hit (-18.9%) while large objects are essentially unaffected (+4.6%, within noise) -- 8-bit resolution has the least room to spare exactly where the signal is already thinnest.

Table 4. mAP@0.5 across all five conditions - FP32 vs. INT8.

Condition	FP32	INT8	INT8 relative gap
Low light ($\gamma = 2.5$ gamma correction)	0.516	0.512	-0.6%
Clean (no degradation)	0.586	0.569	-3.0%
JPEG compression, quality 30	0.534	0.514	-3.6%
Downscale to 50%, then upscale back	0.543	0.517	-4.8%
Motion blur (15×15 horizontal kernel)	0.281	0.250	-10.9%

"INT8 relative gap" is INT8's shortfall as a fraction of FP32's own score in that condition, so it's comparable across rows even though FP32 itself collapses under motion blur. Reading the table: INT8's accuracy penalty relative to FP32 does **not** grow uniformly with how visually damaging the degradation looks — it barely moves under low light, but nearly quadruples under motion blur, which is also the single worst condition for both models by far.

Table 5. Task 3 intervention — recalibrating INT8 on blurred images.

Model	mAP@0.5 on motion blur	mAP@0.5 on clean images
FP32	0.281	0.586
INT8, clean-calibrated (original)	0.250	0.569
INT8, blur-calibrated (new)	0.250	0.582

The intervention **did not** close the gap on its intended target (motion blur: 0.250 → 0.250, no real change). It did, unexpectedly, close 78% of the original gap on clean images instead (0.569 → 0.582), recalibrating on blurred data made 95% of the model's 124

quantization scales tighter, which improved resolution for the clean distribution's typical activation values but did nothing for information already destroyed by the blur itself. Cost: +6 bytes, <0.3% latency difference, essentially free, ineffective for the stated goal.

What surprised us

- **INT8 was initially slower than FP32** (177 ms vs. 135 ms/image) because the model used **S8S8 quantization**, which misses ONNX Runtime's optimized x86 INT8 kernels. Switching to **U8S8** restored the expected ~2× speedup.
- **Quantizing the output Concat layer broke detection accuracy.** A single quantization scale was shared between bounding box coordinates (0–640) and confidence scores (0–1), causing small confidence values to round to zero and eliminating detections.
- **INT8 accuracy loss depends on the type of degradation, not its visual severity.** Low light had little additional impact, while motion blur significantly increased the FP32–INT8 accuracy gap by removing fine edge details.
- **Recalibrating on blurred images did not improve motion-blur robustness.** Instead, it improved performance on clean images, showing that calibration redistributes available INT8 precision rather than recovering information lost to degradation.

What we'd do with another week

- **Use mixed precision instead of recalibration.** Keep early backbone layers in FP32, where motion blur removes critical edge information, while quantizing the rest of the network.
- **Apply QAT with motion-blur augmentation.** Let the model learn blur-robust weights during training rather than relying solely on post-training calibration.
- **Expand evaluation to the full 5-class COCO val2017 set (~3,000 images).** This would reduce statistical noise in classes with few samples, such as *stop sign*.
- **Evaluate additional real-world degradations and alternative backends.** Test sensor noise, low resolution, and weather effects, and compare ONNX Runtime INT8 against OpenVINO INT8 on the same CPU.